

总体路线概览

- 【须知】 提问须知（先看这个）

本学习路线覆盖的内容都是众所周知、盖棺定论的知识，工具怎么装、函数怎么用、报错怎么解，网上和 AI 里都有现成答案，没有任何提问的价值。

遇到问题（30 分钟原则）：先搜索 → 再问 AI → 最后才考虑提问，提问时必须证明你已经试过、搜过、问过 AI。

唯一可以提问的情况：✓

当你完成了这部分内容、想知道下一步学习方向，比如某一小节做完了，来问"接下来学什么"。

× 文档里写过的内容，不要问 · × 搜一下就有答案的，不要问 · ×

甩一段代码说"帮我看看哪里错了"，不要问

老队员更愿意帮"自己先试过的人"，而不是"伸手党"。

【注意】 重要：所有开发环境（Python / Anaconda / VSCode / OpenCV / Git 等）都要自己准备、自己安装配置，这是基本功，不会有人帮你搭环境。遇到安装配置问题先自己搜索、问 AI，自己解决。

【提示】 建议准备一个网络代理 / 加速工具：国内访问 GitHub 不稳定，git push、git clone、下载依赖经常卡死或失败，自备一个代理或加速工具会简单很多。正经的说法叫网络代理，常用方案有： - Watt Toolkit（原 Steam++）：一键加速 GitHub，免费，最简单 - ghproxy 等镜像站：给 clone 链接加个前缀走镜像，临时救急 - 各类代理工具：配置好后 Git 全走代理，最稳定

注：有 * 标识的表示需要了解，但不会具体的严格考察

工具链搭建 → 语言基础(Python/C++二选一) → OpenCV入门 → 图像处理与目标提取 → 实战项目 → 注册GitHub并上传项目

- 先说结论：Python 和 C++ 都可以学，选一门作为主力语言即可，但力扣刷题是必须的。
- Python：上手快、写起来舒服，适合快速验证算法，队内做算法验证、数据集处理基本都用 Python
 - C++：真正部署到机器人上的视觉代码基本都是 C++（CMake 编译、串口通信、性能要求高）
 - 建议：入门先选 Python，学有余力再补 C++，两个方向后续都会用到
 - 力扣刷题：无论选哪门语言都要刷，这是编程基本功的硬指标，训练逻辑思维，后期处理轮廓匹配、坐标变换、滤波通

一、工具链搭建

1.1 开发环境

工具	用途	备注
Python 3.10+	主力语言	推荐用 Anaconda 管理环境

VSCode	代码编辑	装 Python 插件，顺带学 Git
Anaconda	环境管理	conda 创建虚拟环境，避免库版本互相污染
OpenCV	图像处理库	本阶段的核心库
Git	版本控制	团队协作必备，上传项目要用
CMake（C++ 方向）	构建工具	学 C++ 再接触，先了解

Python 环境安装示例：

```
conda create -n vision python=3.10
conda activate vision
pip install opencv-python numpy
```

自检：能用 `cv2.imread()` 读一张图片并 `cv2.imshow()` 显示出来，能打开摄像头，环境就算搭好了。

1.2 硬件工具（了解）

- ☐ USB 摄像头 / CSI 摄像头（先用电脑自带的就行）
- ☐ 一张打印出来的红蓝光条图案（或直接截图当素材）

二、编程语言基础

目标：能独立写出完成简单任务的脚本，掌握数据结构与函数，养成刷题习惯。

2.1 Python 基础（选 Python 的同学）

- ☐ 变量、数据类型、运算符
- ☐ 流程控制：if、for、while
- ☐ 列表、元组、字典、集合（增删改查）
- ☐ 函数定义与作用域
- ☐ 面向对象入门：类、对象、方法
- ☐ NumPy 基础：ndarray、切片、索引、广播 —— OpenCV 读进来的图像本质上就是一个 NumPy 数组，这个必须熟悉

2.2 C++ 基础（选 C++ 的同学）

- ☐ 语法、循环、条件
- ☐ 数组、指针、引用
- ☐ STL 容器：vector、map、pair
- ☐ 类与对象、命名空间
- ☐ 编译流程：g++ / CMake 基本使用

2.3 力扣刷题（必做）

为什么刷力扣：视觉组后期要处理大量算法问题——轮廓匹配、坐标变换、卡尔曼滤波、最小外接矩形、凸包等，背后全

- ☐ 平台：[力扣 LeetCode](#)（也可用洛谷补充）
- ☐ 刷题顺序：LeetCode 热题 100 或 剑指 Offer，从简单题开始
- ☐ 频率建议：每周 3-5 题，先自己想 20-30 分钟，想不出再看题解
- ☐ 用什么语言刷都行，用你选的主力语言刷
- ☐ 重点题型：数组、双指针、哈希表、链表、二叉树、排序
- ☐ 进阶题型：二分、回溯、动态规划（简单-中等难度）

要求：能独立完成力扣简单题，能看懂并复现中等题题解

推荐资源： - [Python 官方教程（中文）](#) - [廖雪峰 Python 教程](#) - 黑马程序员 C++ 教程（B 站搜即可） - [cppreference.com](#)（C++ 官方参考文档） - [力扣 LeetCode](#)

三、OpenCV 入门

目标：掌握图像处理基础操作，能对一张图片做各种处理并实时可视化结果。

3.1 图像读取与显示

- ☐ imread / imshow / imwrite
- ☐ BGR 与 RGB 的颜色顺序差异（OpenCV 默认 BGR，容易踩坑）
- ☐ 像素访问、ROI（感兴趣区域）裁剪

3.2 视频与摄像头读取（实战必用）

- ☐ VideoCapture：cap = cv2.VideoCapture(0) 打开摄像头，cap.read() 读一帧
- ☐ waitKey() 控制刷新与退出，cap.release() 释放摄像头
- ☐ 视频本质就是连续帧，while 循环里逐帧处理即可
- ☐ 实战：读摄像头并实时显示画面

```
import cv2

cap = cv2.VideoCapture(0)          # 0 = 电脑自带摄像头；图片/视频路径也可以
while True:
    ret, frame = cap.read()        # ret: 是否读到帧, frame: 一帧图像
    if not ret:
        break
    cv2.imshow("frame", frame)     # 显示这一帧
    if cv2.waitKey(1) & 0xFF == ord('q'): # 按 q 退出
        break

cap.release()                      # 释放摄像头
cv2.destroyAllWindows()
```

3.3 颜色与阈值

- ☐ 颜色空间转换: `cvtColor` (BGR → HSV 等)
- ☐ HSV 阈值提取: `inRange`
- ☐ 二值化: `threshold`、自适应阈值 `adaptiveThreshold`
- ☐ 实战: 提取画面中的红色光条 / 蓝色光条 (模拟装甲板灯条)

3.4 形态学操作

- ☐ 腐蚀 / 膨胀: `erode` / `dilate`
- ☐ 开运算 / 闭运算: `morphologyEx`
- ☐ 实战: 去掉阈值提取后的噪点

3.5 边缘检测与轮廓查找 (核心)

- ☐ 高斯滤波、Canny 边缘检测
- ☐ `findContours` 轮廓查找 —— 视觉组最常用的函数, 没有之一
- ☐ 轮廓属性: `boundingRect`、`minAreaRect` (最小外接矩形)、面积、周长、凸度
- ☐ 按面积、宽高比、角度筛选轮廓, 滤掉干扰
- ☐ 实战: 找出画面中所有候选灯条并画框

3.6 直线与形状检测

- ☐ 霍夫直线检测: `HoughLinesP`
- ☐ 多边形拟合: `approxPolyDP`
- ☐ 实战: 识别类圆形 / 类矩形目标

3.7 模板匹配与特征匹配 *

- ☐ `matchTemplate` 模板匹配
- ☐ ORB / SIFT 特征点匹配 (了解即可)

要求: 能独立写一个程序, 从摄像头或图片中稳定识别出指定颜色的目标物体, 并用矩形框出来、输出中心坐标

推荐资源: - [OpenCV-Python 官方教程](#) - B 站「唐宇迪 OpenCV」系列 (经典, 搜索即得) - 官方例程: 安装 OpenCV 后的 `samples/` 目录 - 动手查 `cv2` 各函数的官方文档, 养成查文档的习惯

四、实战项目

说明: 三选一, 做出来的项目最终要上传到自己的 GitHub 仓库 (见下一节)。做的时候想清楚每一步为什么这么做, 不要照抄代码。

- 颜色目标识别: 识别画面中红色/蓝色的光条 (模拟装甲板灯条), 框出来并输出中心坐标与角度
- 简易装甲板识别: 识别一对灯条 → 匹配成装甲板 → 输出中心坐标 (进阶, 最接近组内实际任务)
- 人脸/圆形检测: 用 Haar / Hough 检测固定形状目标

最终要求： - 能读摄像头实时画面并实时处理（帧率别太低） - 代码注释完整、结构清晰、参数写清楚 - 写进自己的 GitHub 仓库（见下一节）

五、Git 与 GitHub

目标：注册 GitHub 账号，创建仓库，把上面的 OpenCV 项目传上去，掌握最基本的版本控制流程。

5.1 注册 GitHub

1. 打开 github.com，用邮箱注册账号并完成邮箱验证
2. （可选）配置 SSH key，或用 HTTPS 方式上传

5.2 创建仓库

1. 点击 New repository：起一个清晰的名字（如 armor-detection）、勾选添加 README、选择开源协议（如 MIT）
2. git clone 到本地

5.3 上传项目

```
git init
git add .
git commit -m "feat: 颜色目标识别初版"
git branch -M main
git remote add origin https://github.com/<你的用户名>/<仓库名>.git
git push -u origin main
```

5.4 提交规范

1. 提交信息写清楚：feat: fix: docs: 前缀 + 一句话说明，如 "feat: 添加灯条筛选" 优于 "更新"
2. 不要上传：__pycache__、.pyc、.venv、大图片素材等 → 用 .gitignore 排除
3. 常用命令：git status / git add / git commit / git push / git pull
4. 推送前先 git pull，避免冲突

最终要求： 1. ✓ 注册 GitHub 账号 2. ✓ 上传一个自己写的 OpenCV 项目（哪怕很简单） 3. ✓ 仓库里有 README，写清楚项目是干什么的、怎么运行 4. ✓ 能独立完成 clone / add / commit / push / pull

推荐资源： - [GitHub 官方文档](#) - [Git 教程](#) - [廖雪峰](#) - B 站搜「Git 入门 10 分钟」，看一遍上手即可

六、AI 使用规范

AI 可以用，而且鼓励用。但你要清楚考核只看两件事：① 你自己编写代码的能力；② 项目的最终效果。

6.1 考核到底考什么

考核项	内容	对应要求
-----	----	------

① 自己写代码	现场/限时编写代码的能力	禁止使用 AI
② 项目最终效果	项目跑起来的效果	允许用 AI，但看的是整个体系的稳定性和开发效率

6.2 为什么不能无脑用 AI

场景	后果
考核时自己写不出代码	第 ① 项直接挂
AI 生成的代码不调试、不测试直接上	系统不稳定，第 ② 项挂
依赖 AI 凑功能，不理解原理	出 bug 无法排查，效率反而更低

核心：AI 可以帮你提速，但体系的稳定性和开发效率才是底线。无脑用 AI——不调试、不理解、不测试——项目一定跑不起来，考核一定过不了。

七、标准

- 1. 代码规范：命名清晰、注释完整、模块化
- 2. 动手能力：能独立完成一个从摄像头采集到结果输出的完整视觉流程
- 3. 团队协作：Git 提交规范、及时同步进度
- 4. 自主排查：遇到问题先自己解决，不依赖别人喂答案